

CQRS Nedir ve Avantajları

CineSeat Projesi Özelinde

CQRS'in ne olduğu, CineSeat'e ne kazandırdığı ve bedelinin ne olduğu; tamamen CineSeat'in kendi gerçek kodundan örneklerle.

*Ekip: Kaan Sezer · Alptuğ Dursun · Ömer Faruk Çendek · İzzettin Berke Kuş
Ağustos 2026*

İçindekiler

1. CQRS Nedir?	3
1.1 Açılımı ve kökeni	3
1.2 Temel fikir, tek cümlede	3
1.3 Yaygın yanlış anlamalar	3
2. CQRS Olmadan: Klasik Yaklaşımın Sorunu	5
3. CQRS'in Yapı Taşları (CineSeat'te)	6
4. CQRS'in Avantajları	7
4.1 Tek sorumluluk, küçük ve okunabilir handler'lar	7
4.2 Okuma ve yazma birbirinden bağımsız optimize edilebilir	7
4.3 Yeni özellik eklemek mevcut kodu bozmaz (Open/Closed)	7
4.4 Cross-cutting concern'ler tek yerde, otomatik	8
4.5 Test edilebilirlik	8
4.6 Okunabilirlik: "Bu iş nerede yazılı?" sorusuna tek cevap	8
4.7 Ekip paralelliği	9
5. Bedeli Ne? (Dengeli Bakış)	10
6. Özet: CQRS Var / CQRS Yok	11
Sonuç	11

1. CQRS Nedir?

1.1 Açılımı ve kökeni

CQRS = *Command Query Responsibility Segregation* — Türkçesi: **Komut-Sorgu Sorumluluk Ayrımı**. Bir yazılım sisteminde **veriyi değiştiren işlemler** (komutlar) ile **veriyi okuyan işlemler** (sorgular) için ayrı modeller, ayrı sınıflar, ayrı akışlar kullanma prensibidir.

Fikrin atası, Bertrand Meyer'in 1980'lerde ortaya attığı **CQS** (Command Query Separation) ilkesidir: *bir metot ya durumu değiştirir (command) ya veri döndürür (query), ikisini birden yapmaz*. 2010 yılında Greg Young bu fikri metot seviyesinden **mimari seviyesine** taşıdı ve CQRS oldu: okuma ve yazma sadece aynı metotta değil, **tamamen ayrı sınıflarda, ayrı modellerde** yaşasın.

Gerçek hayat karşılığı — Sinema gişesi

Gişede sipariş almak (bir bilet satmak) ile menü/seans tablosunu göstermek birbirinden çok farklı işlerdir. Sipariş alırken (**command**) titiz olmak zorundasın: koltuk müsait mi, ödeme doğru mu, kampanya geçerli mi — yavaş ama kesin olmalı. Seans tablosunu gösterirken (**query**) hızlı olmalısın — her müşteri için mutfağa/depoya gidip tek tek sormazsın, elindeki listeyi olduğu gibi gösterirsin.

1.2 Temel fikir, tek cümlede

Tanım

Bir sistemde veriyi **değiştirmek** için kullanılan yol (Command → Handler → Entity) ile veriyi **okumak** için kullanılan yol (Query → Handler → DTO) birbirinden tamamen bağımsızdır; aynı sınıf, aynı metot, hatta çoğu zaman aynı doğrulama/iş kuralı seti bile paylaşılmaz.

CineSeat'te bu ayrım klasör seviyesinde bile görünür — her özellik modülünün altında Commands/ ve Queries/ iki ayrı klasördür ve birbirinin sınıflarını hiçbir zaman kullanmaz:

```
Features/Movies/  
  Commands/  
    CreateMovie/    CreateMovieCommand, CreateMovieCommandHandler, CreateMovieCommandValidator  
    UpdateMovie/    UpdateMovieCommand, UpdateMovieCommandHandler, UpdateMovieCommandValidator  
    DeleteMovie/    DeleteMovieCommand, DeleteMovieCommandHandler  
  Queries/  
    GetMovies/      GetMoviesQuery, GetMoviesQueryHandler      (liste + arama + sayfalama)  
    GetMovieById/   GetMovieByIdQuery, GetMovieByIdQueryHandler    (tek film detayı)  
  DTOs/  
    MovieDto
```

backend/Core/CineSeat.Application/Features/Movies/ (gerçek klasör yapısı)

CreateMovieCommand bir film oluşturur ve geriye sadece long Id döner; GetMoviesQuery aynı Movie tablosunu okur ama PagedResult<MovieDto> döner. İkisi **aynı tabloya, farklı sınıflarla, farklı handler'larla** gider.

1.3 Yaygın yanlış anlamalar

- "CQRS iki ayrı veritabanı demektir" — Yanlış. CineSeat tek bir PostgreSQL veritabanı, tek bir ApplicationDbContext kullanıyor. Ayrılan şey **model**dir (yazarken entity, okurken DTO), veri deposu değil.

- **"CQRS için Event Sourcing zorunludur"** — Yanlış. İkisi sık birlikte anılır ama bağımsız kavramlardır. CineSeat CQRS kullanıyor, Event Sourcing kullanmıyor.
- **"CQRS her projede gerekir"** — Yanlış. Basit CRUD ekranlarında (CineSeat'teki Genres, Technologies gibi) CQRS'in getirdiği ayrım neredeyse görünmez — command ve query'nin ikisi de tek satırlık bir işlem. Bölüm 5'te bu dengeyi ayrıca ele alıyoruz.

2. CQRS Olmadan: Klasik Yaklaşımın Sorunu

CQRS'in değerini anlamamanın en kolay yolu, onsuz nasıl yazılacağını görmek. Klasik (CQRS'siz) bir yaklaşımda tek bir "servis" sınıfı hem okuma hem yazma işlerini üstlenir:

```
// KLASİK YAKLAŞIM (CineSeat'te KULLANILMIYOR -- karşılaştırma için)
public class MovieService
{
    private readonly AppDbContext _context;

    public Movie GetById(long id) => _context.Movies
        .Include(m => m.Comments).Include(m => m.MovieGenres)
        .Include(m => m.Showtimes).Include(m => m.UserFavorites)
        .First(m => m.Id == id); // Liste ekranı için de AYNI metot çağrılır

    public List<Movie> GetAll() => _context.Movies
        .Include(m => m.Comments).Include(m => m.MovieGenres) // Gerekmese de hep yüklenir
        .ToList();

    public void Create(Movie movie) { _context.Movies.Add(movie); _context.SaveChanges(); }
    public void Update(Movie movie) { _context.Movies.Update(movie); _context.SaveChanges(); }
    public void Delete(long id) { ... }
}
```

Karşılaştırma amaçlı örnek — CineSeat'in gerçek kodu değildir

Bu yaklaşımın üç somut sorunu var:

- **Aşırı veri çekme.** Film listesi ekranı sadece başlık, süre ve afiş gösterecekken, GetAll() her filmin tüm yorumlarını, tüm seanslarını da veritabanından çeker — CineSeat ölçeğinde (yüzlerce film, binlerce yorum) bu saniyeler sürebilecek bir israf olurdu.
- **Şişen, karışık sınıf.** MovieService büyüdükçe içine "favorilere ekle", "puan hesapla", "tür ata" gibi ilgisiz işler de eklenmeye başlar — tek sınıf, tek dosya, onlarca sorumluluk.
- **Okuma ile yazmanın kuralları birbirine karışır.** Bir filmi oluştururken uygulanması gereken doğrulama (ageLimit pozitif mi, başlık boş mu) ile bir filmi listelerken uygulanması gereken mantık (arama filtresi, sayfalama) aynı sınıfın içinde iç içe geçer.

CQRS bu üçünü de doğrudan çözer: her use case kendi dosyasında, kendi modeliyle, kendi kurallarıyla yaşar (bkz. Bölüm 3-4).

3. CQRS'in Yapı Taşları (CineSeat'te)

Bileşen	Görevi	CineSeat örneği
Command	Durumu değiştirme isteği	CreateReservationCommand, AddCommentCommand
Query	Veri okuma isteği	GetMoviesQuery, GetCinemasByCityQuery
Handler	Bir Command/Query'yi uçtan uca yürüten sınıf	CreateReservationCommandHandler
Mediator (MediatR)	Controller'ı handler'dan haberdar etmeden araya giren aracı	IMediator.Send(command)
Validator	Command/Query'nin şeklen doğruluğunu kontrol eder	CreateCinemaCommandValidator
Pipeline Behavior	Her Command/Query'nin geçtiği ortak halka (loglama, doğrulama)	LoggingBehavior, ValidationBehavior
DTO	Query'nin dışarıya verdiği, sunuma hazır veri	MovieDto, ReservationDto

Bir controller, hangi handler'ın çalışacağını bilmez — sadece "bu isteği hallet" der:

```
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateCinemaCommand command)
    => Ok(await _mediator.Send(command));

[HttpGet]
public async Task<IActionResult> GetByCity([FromQuery] long cityId, ...)
    => Ok(await _mediator.Send(new GetCinemasByCityQuery { CityId = cityId, ... }));
```

backend/Presentation/CineSeat.WebAPI/Controllers/CinemasController.cs (gerçek kod)

Create bir CreateCinemaCommand alıp long döner; GetByCity bir GetCinemasByCityQuery alıp PagedResult<CinemaDto> döner. Aynı controller, iki farklı dünyaya aynı tek satırla (_mediator.Send(...)) kapı açıyor.

4. CQRS'in Avantajları

4.1 Tek sorumluluk, küçük ve okunabilir handler'lar

Her handler tek bir işi yapar. CineSeat'te sağlıklı bir handler genelde 15-30 satırdır — CreateCinemaCommandHandler üç adımdan ibarettir: **kontrol et → oluştur → kaydet**:

```
public async Task<long> Handle(CreateCinemaCommand request, CancellationToken ct)
{
    var district = await _districtRead.GetByIdAsync(request.DistrictId, tracking: false, ct);
    if (district is null) throw new NotFoundException("İlçe", request.DistrictId);

    var cinema = new Cinema { Name = request.Name, Address = request.Address, ... };

    await _write.AddAsync(cinema, ct);
    await _write.SaveAsync(ct);
    return cinema.Id;
}
```

backend/.../CreateCinemaCommandHandler.cs (gerçek kod, kısaltılmış)

200 satırlık, on farklı işi birden yapan bir "tanrı sınıfı" yerine, CineSeat'te ~40 küçük, ismi işini anlatan handler var. Yeni bir geliştirici CreateCinemaCommandHandler.cs dosyasını açtığında tam olarak ne yaptığını 30 saniyede anlar.

4.2 Okuma ve yazma birbirinden bağımsız optimize edilebilir

Yazma tarafı bütünlük ve iş kuralı ister; okuma tarafı hız ve az veri ister. CQRS bu iki farklı ihtiyacın birbirine karışmasını engeller. GetMoviesQueryHandler, sadece ihtiyaç duyulan 9 kolonu SQL'e taşıyan bir **projeksiyon** kullanıyor:

```
.Select(m => new MovieDto
{
    Id = m.Id, Title = m.Title, Duration = m.Duration, Poster = m.Poster,
    StartDate = m.StartDate, EndDate = m.EndDate, AvgScore = m.AvgScore, ...
}); // EF Core bunu görünce sadece bu 9 kolonu SELECT eder -- Comments,
// Showtimes, UserFavorites gibi ilişkili koleksiyonlar HİÇ çekilmez.
```

backend/.../GetMoviesQueryHandler.cs (gerçek kod)

Aynı Movie tablosuna yazarken ise CreateMovieCommandHandler tamamen farklı bir kaygıyla çalışır: doğru EndDate/StartDate girildi mi, AgeLimit geçerli mi. İki taraf da kendi işine odaklanır, birbirinin performans/doğruluk ihtiyacını taşımaz.

İleri potansiyel

Bugün CineSeat tek veritabanı kullanıyor, ama CQRS ayrımı sayesinde ileride sadece **okuma tarafını** ayrı ölçeklendirmek (cache eklemek, salt-okunur bir read-replica'ya yönlendirmek) mümkün — çünkü query'ler zaten yazma modelinden bağımsız. Bu, CQRS kullanmayan bir sistemde çok daha büyük bir yeniden yazım gerektirir.

4.3 Yeni özellik eklemek mevcut kodu bozmaz (Open/Closed)

CineSeat'te bugün ~40 use case var. Örneğin "favorilere ekle" özelliği eklendiğinde MoviesController'a, GetMoviesQueryHandler'a ya da başka hiçbir mevcut dosyaya dokunulmadı — sadece yeni bir klasör (Features/UserFavorites/) ve içine yeni Command/Query/Handler dosyaları eklendi. Mevcut 39 use case'in

hiçbiri risk altına girmedir.

Neden önemli?

Klasik "tek servis" yaklaşımında (MovieService örneği, Bölüm 2) yeni bir metod eklemek, aynı sınıfın içine dokunmak demektir — o sınıfı kullanan HER YER (10 farklı controller olabilir) potansiyel risk altındadır. CQRS'te yeni bir Command/Query, kendi izole dosyasında yaşar; onu hiç çağırmayan hiçbir kod etkilenmez.

4.4 Cross-cutting concern'ler tek yerde, otomatik

Loglama ve doğrulama gibi "her isteğin ihtiyaç duyduğu ama iş mantığıyla ilgisi olmayan" işler, CQRS'in **pipeline behavior** mekanizmasıyla tek yerde toplanır ve tüm 40 handler'a otomatik uygulanır — hiçbirine elle eklenmez:

```
services.AddTransient(typeof(IPipelineBehavior<, >), typeof(LoggingBehavior<, >));
services.AddTransient(typeof(IPipelineBehavior<, >), typeof(ValidationBehavior<, >));
```

backend/Core/CineSeat.Application/DependencyInjection.cs (gerçek kod)

Bu iki satır sayesinde CineSeat'teki **hiçbir** handler kendi içinde try/catch ile doğrulama yapmıyor, kendi içinde `_logger.Log(...)` çağırıyor. `ValidationBehavior`, handler çalışmadan önce ilgili `AbstractValidator<T>`'ı otomatik bulup çalıştırıyor; hata varsa handler'a hiç girilmiyor.

4.5 Test edilebilirlik

Bir handler'ın bağımlılıkları (repository'ler, `ICurrentUserService`) hepsi arayüz üzerinden constructor'a enjekte ediliyor. Bu, bir handler'ı test ederken gerçek veritabanına hiç dokunmadan, sahte (mock) repository'lerle test edebilmek demektir:

```
// Örnek -- CineSeat'te henüz bir test projesi yok ama mimari buna hazır
var fakeWrite = new Mock<ICinemaWriteRepository>();
var fakeDistrictRead = new Mock<IDistrictReadRepository>();
fakeDistrictRead.Setup(r => r.GetByIdAsync(1, false, default)).ReturnsAsync(new District { Id = 1
});

var handler = new CreateCinemaCommandHandler(fakeWrite.Object, fakeDistrictRead.Object);
var id = await handler.Handle(new CreateCinemaCommand { DistrictId = 1, Name = "Test" }, default);

fakeWrite.Verify(w => w.AddAsync(It.IsAny<Cinema>(), default), Times.Once);
```

Kavramsal örnek — repository'ler arayüz olduğu için mock'lanabilir

Bu, tek büyük `MovieService` sınıfını test etmekten çok daha kolaydır — orada tüm bağımlılıkları (tam bir `DbContext` dahil) ayağa kaldırmadan tek bir metodu izole test etmek zordur.

4.6 Okunabilirlik: "Bu iş nerede yazılı?" sorusuna tek cevap

CQRS, `CineSeat`'in `Features/` klasörü yapısıyla birleşince *Screaming Architecture* ilkesini doğal olarak uyguluyor: mimari "ben `CineSeat.WebAPI`'yim" değil, "ben rezervasyon alıyorum, film listeliyorum, yorum ekliyorum" diye bağırıyor:

"Rezervasyon iptali nerede?"	-> <code>Features/Reservations/Commands/CancelReservation/</code>
"Film arama nerede?"	-> <code>Features/Movies/Queries/GetMovies/</code>
"Kampanya indirimi nerede?"	-> <code>Features/Reservations/Commands/CreateReservation/</code> (indirim hesabı burada)

407 dosyalık bir kod tabanında bu, yeni bir ekip üyesinin ilk günde yön bulabilmesi için ölçülebilir bir fark yaratıyor.

4.7 Ekip paralelliği

CQRS'in her use case'i kendi izole dosya grubunda yaşadığı için, birden fazla geliştirici aynı anda **farklı** Command/Query'ler üzerinde çalışırken birbirinin koduna neredeyse hiç dokunmaz. CineSeat 4 kişilik bir ekip tarafından geliştiriliyor — biri Reservations modülünde, biri Movies modülünde çalışırken merge çakışması riski, tek bir dev MovieService üzerinde çalışmaya kıyasla belirgin biçimde düşüktür.

5. Bedeli Ne? (Dengeli Bakış)

CQRS bedavaya gelmiyor — dürüst olmak gerekirse bu bedelleri de görmek gerekir:

- **Daha fazla dosya.** Tek bir "film" özelliği için CineSeat'te Command + Handler + Validator + Query + Handler + DTO gibi 6+ dosya var. Klasik yaklaşımda bu tek bir `MovieService.cs` olurdu.
- **Basit CRUD'da abartı olabilir.** Genres, Technologies gibi gerçekten basit modüllerde bu ayrımın getirdiği fayda neredeyse sıfır — CineSeat bu modüllerde bilerek validator bile yazmamış, sadeleştirmiş.
- **Öğrenme eğrisi.** Yeni bir geliştirici için "neden bu kadar çok küçük dosya var" sorusu ilk günlerde kafa karıştırabilir.

CineSeat'in dengesi

Karmaşıklığı ihtiyaç duyulmadan eklememek CQRS'in kendisiyle çalışmıyor — aynı prensip her modülde aynı titizlikte uygulanmak zorunda değil. CineSeat'te karmaşık modüller (Reservations) tam CQRS disipliniyle, basit modüller (Cities) daha hafif yazılmış. Bu, "CQRS'i yanlış kullanmak" değil, ölçeğe göre ayarlamaktır.

6. Özet: CQRS Var / CQRS Yok

Konu	CQRS olmadan (tek servis)	CQRS ile (CineSeat)
Okuma sorgusu	Tüm entity + ilişkiler çekilir	Sadece gereken kolonlar (Select projeksiyonu)
Yeni özellik eklemek	Mevcut büyük sınıfı değiştirmek gerekir	Yeni, izole bir dosya grubu eklenir
Loglama / doğrulama	Her metotta elle tekrarlanır	Pipeline behavior ile otomatik, tek yerde
Test etmek	Büyük sınıfı ayağa kaldırmak gerekir	Tek handler, mock repository ile izole test
"Bu iş nerede?" sorusu	Büyük dosyada arama yapmak gerekir	Features/X/Commands Queries/Y/ — isimden belli
Ekip çalışması	Aynı dosyada çakışma riski yüksek	Farklı modüllerde paralel, düşük çakışma
Basit CRUD maliyeti	Düşük (zaten tek metot)	Gereksiz olabilir — CineSeat bu modüllerde sadeleştiriyor

Sonuç

CQRS, CineSeat'in ~40 use case'ini, 4 kişilik bir ekibin paralel çalışabileceği, her birinin nerede yaşadığı belli, okuma performansı yazma iş kurallarından bağımsız optimize edilebilen küçük parçalara bölmemizi sağladı. Bedeli daha fazla dosya ve biraz daha fazla ilk kurulum emeğiydi — karşılığında aldığımız şey, her mimari kararın aslında cevapladığı o kilit soruya ("bir şeyi değiştirdiğimde başka neyi değiştirmek zorunda kalacağım?") çoğu zaman "hiçbir şeyi" cevabını verebilmek.